

Recommender Systems: from Collaborative Filtering to Neural CTR to DLRM

WEC NITK Intelligence SIG Recruitment 2026 — Mandatory Task Report

Ishaan · Mining Engineering, NITK Surathkal · GitHub: vector298

October 2026

Summary

This report covers the three recommender subtasks: (01) memory-based collaborative filtering against matrix factorisation written from scratch, on MovieLens 1M; (02) a neural click-through-rate (CTR) model on the supplied advertising data; and (03) a from-scratch Deep Learning Recommendation Model (DLRM) on the same CTR data and split, with an ablation of its interaction layer. Every number below comes from my own notebook outputs.

- **Task 01.** Matrix factorisation was the best rating predictor (test RMSE 0.8471), ahead of item-item CF (0.8523) by a small margin that is outside bootstrap noise (paired difference 0.0052, 95% interval [0.0036, 0.0068]). Both beat a bias baseline (0.9031). On top-10 ranking over all unseen movies, a popularity baseline beat every rating-based method.
- **Task 02.** Six MLP architectures were indistinguishable on validation (PR-AUC spread 0.0056, about the size of seed noise). The pre-registered rule picked the smallest (43,457 parameters). Test PR-AUC was 0.0647, clearly below validation (0.1014); the cause was investigated but not established.
- **Task 03.** DLRM gave no measurable gain over Task 02. Replacing its pairwise dot-product interactions with plain concatenation improved validation PR-AUC in all six seed-matched comparisons. The selected DLRM uses about four times the parameters of the Task 02 model.
- **Overall.** In both domains the largest gains came from simple structure (bias terms, embeddings); each more elaborate model added little on these dataset sizes.

1. Common protocol

The same discipline was applied to all three tasks: a documented, seeded split; every preprocessing statistic learned from the training part only; hyperparameters and thresholds chosen on validation; the test set scored once at the end; selection rules fixed before running sweeps; and multiple seeds (42, 43, 44) where single-run noise could plausibly exceed the differences being measured. Uncertainty on test metrics is reported with 95% bootstrap intervals. The supplied `test.csv` for Tasks 02 and 03 was never used for any decision.

2. Task 01: Collaborative Filtering on MovieLens 1M

2.1 Dataset and split

MovieLens 1M (GroupLens; Harper and Konstan, 2015) contains 1,000,209 explicit ratings (1–5) from 6,040 users on 3,706 movies, a density of 4.47%. Every user has at least 20 ratings (median 96, maximum 2,314); items have between 1 and 3,428 (median 123), and 7.8% of items have fewer than 5. Ratings skew positive (mean 3.582; 4 is the most common value at 34.9%). I chose it because explicit ratings let both algorithm families predict the same quantity and be scored by RMSE and MAE, and because it is large enough for stable estimates (about 100,000 ratings each in validation and test) while still small enough for dense matrices. A pilot on MovieLens 100K used the same method; all reported results are on 1M.

The ratings were split randomly 80/10/10 (seed 42) into 800,167 train, 100,020 validation and 100,022 test ratings. No validation user and only 19 validation items (0.02%) are unseen in training; these fall back to the baseline. Limitations of the dataset: explicit rather than implicit feedback, self-selected raters, heavy popularity skew, and small by industrial standards.

2.2 Baselines

Four baselines: global mean, user mean, item mean and a bias baseline $pred = \mu + b_u + b_i$ with biases shrunk by 10. The bias baseline reaches validation RMSE 0.9085, 18.7% below the global mean (1.1175), and is the reference the CF methods must beat.

2.3 Memory-based CF

The prediction is the bias baseline plus a similarity-weighted average of the user's residuals (rating minus baseline) on neighbouring items (item-item) or users (user-user). Similarity is the cosine between residual vectors (missing entries treated as 0), multiplied by a shrinkage factor $n/(n + \lambda)$ so that similarities resting on few co-ratings are discounted. Each row keeps its k most similar neighbours; only those the user has rated contribute, and with none the prediction falls back to the baseline.

The grid covered both variants, k from 50 to 800 and λ in $\{0, 25, 100\}$ (plus 200 and 400 at $k = 200, 400$). Item-item with $k = 200, \lambda = 400$ was selected (validation RMSE 0.8575, effectively tied with $\lambda = 100$ to 400); the best user-user setting was $k = 400, \lambda = 100$ (0.8694). 27 of the first 30 settings beat the baseline, and item-item beat user-user in all 15 matched settings (paired bootstrap difference 0.0117, interval [0.0104, 0.0130]).

2.4 Matrix factorisation from scratch

The model is $pred_{ui} = \mu + b_u + b_i + \mathbf{p}_u \cdot \mathbf{q}_i$, trained by mini-batch SGD with hand-written gradients, L2 regularisation and early stopping on validation RMSE. Of nine configurations (factor size $\times$ regularisation), 64 factors with regularisation 0.05 won (validation RMSE 0.8520, best epoch 25). Regularisation 0.05 was best at every factor size; 0.02 overfitted and 0.1 converged more slowly. Over three seeds the selected configuration averaged 0.8532 (std 0.0008).

2.5 Test results

Method	Test RMSE	95% CI	Test MAE
Global mean	1.1160	[1.1114, 1.1206]	0.9327
Item mean	0.9742	[0.9699, 0.9786]	0.7782
User mean	1.0334	[1.0287, 1.0379]	0.8273
Bias baseline	0.9031	[0.8989, 0.9073]	0.7153
Item-item ($k=200, \lambda=400$)	0.8523	[0.8481, 0.8563]	0.6679
User-user ($k=400, \lambda=100$)	0.8643	[0.8601, 0.8684]	0.6792
Matrix factorisation (64, 0.05)	0.8471	[0.8431, 0.8512]	0.6668

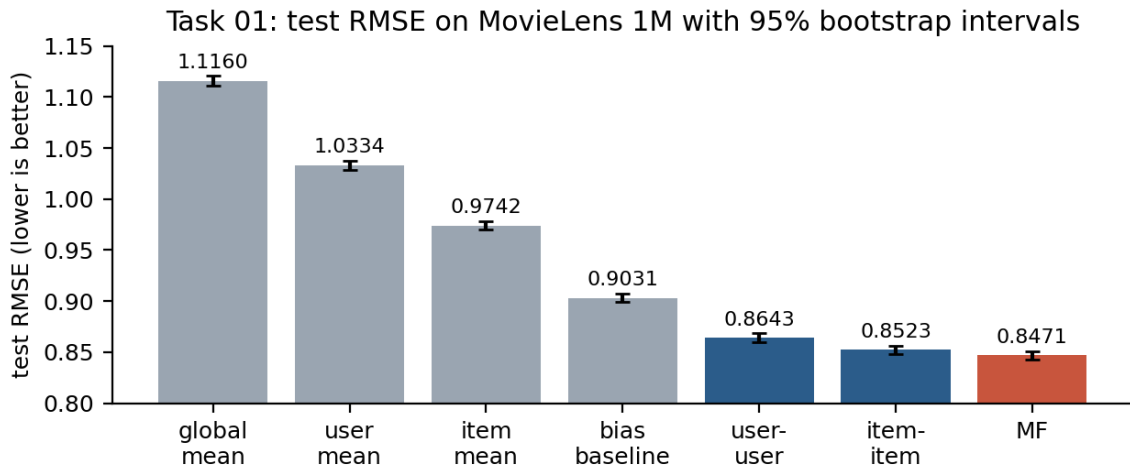


Figure 1. Test RMSE per method. Grey: baselines; blue: memory-based CF; red: matrix factorisation.

MF wins on RMSE by 0.6% over item-item, and the paired bootstrap (item-item minus MF = 0.0052, interval [0.0036, 0.0068]) shows the gap is real rather than sampling noise. The MAE gap, however, is only 0.0011, so the practical difference is small. Relative to the bias baseline, MF reduces test RMSE by 6.2%.

2.6 Ranking view

As a secondary view I ranked all unseen movies for each user by predicted rating and counted ratings of 4 or more in the held-out set as relevant. I also computed a within-user NDCG@10 that only reorders the user's own held-out ratings.

Method	P@10	R@10	NDCG@10	Within-user NDCG@10 (test)
Random	0.0030	0.0033	0.0035	–
Popularity	0.0894	0.0944	0.1201	–
Bias baseline	0.0340	0.0371	0.0450	0.9332
Item-item	0.0271	0.0178	0.0283	0.9443
User-user	0.0001	0.0003	0.0002	0.9419
MF	0.0414	0.0453	0.0528	0.9445

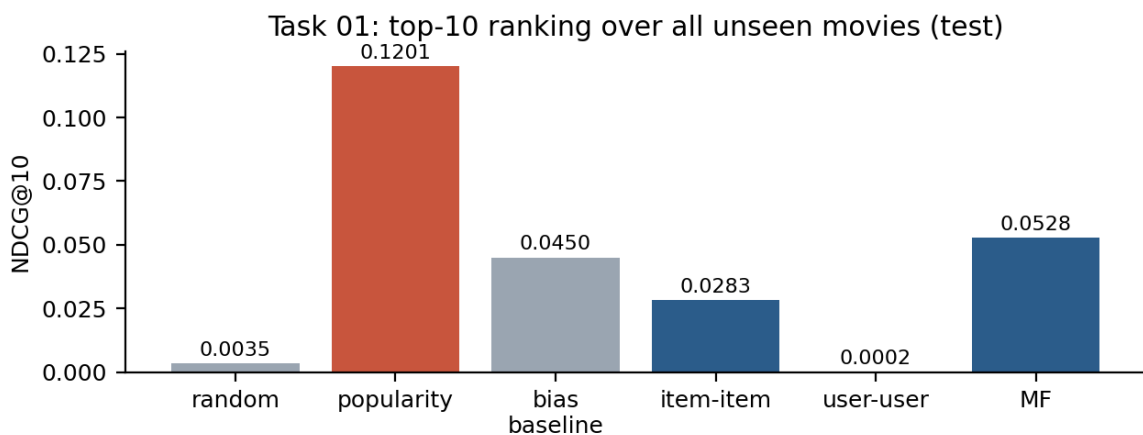


Figure 2. Top-10 NDCG over all unseen movies. Popularity beats every model trained to predict ratings.

Popularity beat every rating-based method on top-10 retrieval. This is a known weakness of ranking by predicted rating: a model optimised for RMSE can assign high predictions to obscure movies the user is unlikely to have seen, while held-out ratings are concentrated on popular titles. My user-user implementation was almost empty of hits; my untested explanation is extreme predictions for movies with very few raters, since the implementation has no damping for that case. On the fairer within-user ordering, item-item and MF tie

(0.9443 and 0.9445), both above the bias baseline (0.9332).

3. Task 02: Neural CTR Prediction

3.1 Data

The supplied data has 40,000 training and 10,000 test impressions, each with 13 integer features, 26 categorical features and a binary click label. Clicks are rare: 1,280 in train (3.2%) and 335 in test (3.35%), so a model that never predicts a click is 96.8% accurate. Accuracy is therefore uninformative; PR-AUC is the primary metric (a random ranker scores about the click rate, 0.032), with ROC-AUC, log loss, F1, precision and recall also reported.

The data is messy: up to about 40% missing values in integer columns and 36% in four categorical columns; categoricals are hashed strings with 3 to 13,508 distinct values (median 2,568); on average 9.2% of non-missing test values never occur in train (up to 36% in one column); integers are heavily skewed (skewness 108 for `integer_feature_1`, maximum 2,068,079); and `integer_feature_8` has 2,455 values of -1.

3.2 Preprocessing (train-only statistics)

`train.csv` was split 80/20, stratified on the label (32,000 train with 1,024 clicks, 8,000 validation with 256 clicks; seed 42). Integers: missing values filled with the training median, negatives clipped to 0 (an assumption, documented), then `log1p` and standardisation. Categoricals: each value maps to an index, with 0 for missing or unseen, 1 for rare values, and one index per frequent value, giving 9,912 embedding rows in total. A split fingerprint (click counts and index sums) is printed so that Task 03 can prove it uses identical data.

3.3 Model and selection

Each categorical column has a learned embedding; embeddings are concatenated with the 13 dense features and passed through an MLP to a single sigmoid output, trained with binary cross-entropy and Adam. Early stopping on validation PR-AUC (patience 3, at most 15 epochs). The baseline (A: 8-dimensional embeddings, 169,153 parameters) overfitted quickly: training loss kept falling while validation loss was lowest at epoch 2 (val PR-AUC 0.1049, log loss 0.1300).

Six configurations were each trained with three seeds. The selection rule, fixed in advance, was: highest mean validation PR-AUC, then the configuration with fewest parameters among those within one standard deviation of it.

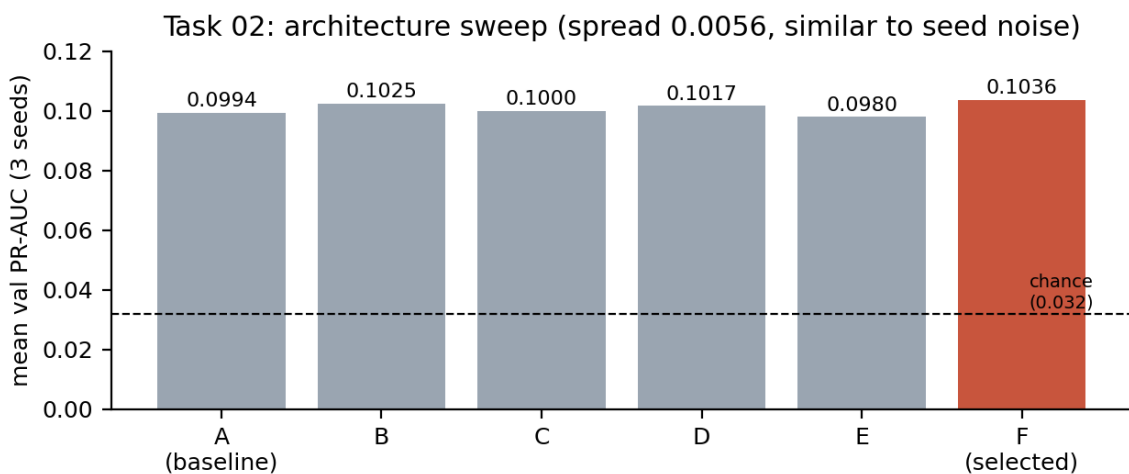


Figure 3. Mean validation PR-AUC over three seeds per architecture. The dashed line is the chance level.

All six configurations fell within one standard deviation of the best (mean PR-AUC 0.0980 to 0.1036; seed standard deviation 0.0006 to 0.0093), so the rule selected the smallest, **F**: 4-dimensional embeddings, one hidden layer of 32 units, no dropout, 43,457 parameters (0.174 MB of weights, about 7 s of CPU training). Its

calibration curve follows the diagonal, with individual bins off by one or two percentage points, which is within noise at roughly 30 clicks per top bin.

3.4 Validation and test

Split	ROC-AUC (95% CI)	PR-AUC (95% CI)	Log loss (95% CI)	F1	Prec.	Recall	Acc.
Val	0.720 [0.688, 0.752]	0.1014 [0.0790, 0.1396]	0.1315 [0.1188, 0.1442]	0.162	0.150	0.176	0.9416
Test	0.668 [0.635, 0.694]	0.0647 [0.0540, 0.0814]	0.1441 [0.1310, 0.1575]	0.085	0.089	0.081	0.9415

Threshold 0.10, chosen to maximise validation F1. Test is a single run.

Test performance is clearly below validation: PR-AUC drops from $3.1\times$ to $1.9\times$ the click rate, and test log loss (0.1441) beats a constant-rate predictor (about 0.1467) by only 0.0026. The drop (0.037 PR-AUC) is far larger than the seed spread, and the bootstrap intervals only just touch. I checked several explanations. The model under-predicts mildly on both splits (mean predicted probability 0.0282 vs observed 0.0320 on validation; 0.0269 vs 0.0335 on test). `test.csv` differs from `train.csv`: missing-rate gaps reach 2.6 percentage points in some integer columns, and 15.7% of test categorical cells fall on the unknown index versus 14.1% on validation. Selection optimism (epoch, configuration and threshold all chosen on validation) probably explains only part of it. **The cause was not established.**

Device effect. The same baseline code and seed gave validation PR-AUC 0.0993 on a Kaggle GPU and 0.1049 on Colab CPU. That 0.0056 difference equals the entire spread across architectures, so single-run differences below about 0.01 are not trusted. All reported Task 02 and Task 03 numbers come from Colab CPU runs.

4. Task 03: DLRM from Scratch

4.1 Architecture

Following Naumov et al. (2019), the 13 dense features pass through a bottom MLP whose output has the same length d as each of the 26 categorical embeddings. All pairwise dot products among these 27 vectors are computed ($27\times 26/2 = 351$ interaction terms), concatenated with the dense vector, and passed through a top MLP to the click probability. The dot product is the same operation as in matrix factorisation (Task 01), now applied across every pair of fields. Task 02's preprocessing, split and selection protocol were reused unchanged.

4.2 Correctness checks

- The split fingerprint printed identically to Task 02 (train clicks 1,024, val clicks 256, categorical index sum 38,103,084), proving identical data and encoding.
- The batched interaction layer was checked against a plain loop over field pairs.
- The parameter count matched a hand calculation (134,409 at $d = 8$).

4.3 Sweep and ablation

The first DLRM run (seed 42) reached only 0.0630 validation PR-AUC and overfitted within three epochs; it later proved to be the weakest of three seeds (mean 0.0735). Eight configurations were then each run with three seeds. P1 and P2 are dot-product models paired with P1c and P2c, which are identical in embedding size, top MLP and dropout but replace the interactions with plain concatenation. D1n repeats the first configuration with PyTorch's default $N(0, 1)$ embedding initialisation instead of $\text{std } 1/\sqrt{d}$.

Config	Interaction	Mean val PR-AUC (3 seeds)	Note
D1	dot	0.0735	first configuration
D1n	dot	0.0687	default $N(0,1)$ init; lower in all 3 seeds
P1	dot	0.0698	$d=4$, top (32), dropout 0

Config	Interaction	Mean val PR-AUC (3 seeds)	Note
P1c	concat	0.0892	same as P1 without interactions
P2	dot	0.0780	d=8, top (64), dropout 0.3
P2c	concat	0.0921	same as P2 without interactions
D4	dot	0.0738	
D5 (selected)	dot	0.0836	d=16, top (64), dropout 0.3; 183,185 params

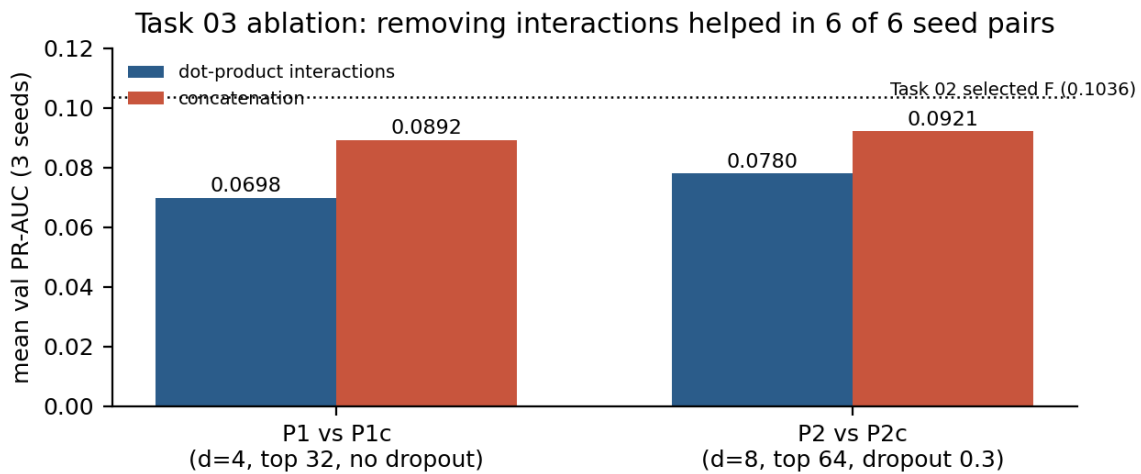


Figure 4. Interaction ablation: dot-product interactions vs concatenation with everything else held fixed. Dotted line: Task 02's selected model.

Concatenation beat dot-product interactions in all six seed-level comparisons, by 0.009 to 0.029 PR-AUC. The initialisation I chose is not the cause, since the default initialisation made DLRM worse in every seed. Among dot-product models, the selection rule picked D5 ($d = 16$, top MLP (64), dropout 0.3). D5 uses the largest d tried, so a better setting may lie beyond the grid.

4.4 Against Task 02

Model	Params	Weights	Val PR-AUC (3 seeds)	Val AUC	Test PR-AUC	Test ROC-AUC
Task 02 baseline A	169,153	0.68 MB	0.0994 (0.0042)	0.7148	–	–
Task 02 selected F	43,457	0.174 MB	0.1036 (0.0075)	0.7187	0.0647	0.668
Task 03 DLRM D5	183,185	0.733 MB	0.0836 (0.0026)	0.7106	0.0665	0.681
Task 03 concat P2c	93,961	0.38 MB	0.0921 (0.0026)	0.7331	–	–

Validation values are three-seed means (std in brackets); test values are single runs. Weight memory = parameters $\times$ 4 bytes.

On validation DLRM is 0.020 PR-AUC below Task 02's model. On test it edges slightly ahead (0.0665 vs 0.0647), but the bootstrap intervals overlap heavily, so the ordering flips within noise generated by only 256 validation and 335 test clicks. The honest conclusion is **no measurable difference**, at roughly four times the parameters and about twice the time per epoch, plus an extra design constraint (the bottom MLP must output exactly d values).

Why might interactions hurt here? I did not test this. Candidate explanations: with only 1,024 training clicks, 351 extra inputs built from sparse, partly rare-valued fields give the network more ways to memorise; dot products of small random embeddings start as noise; and DLRM was designed for datasets many orders of magnitude larger. My own implementation choices (a ReLU on the bottom MLP's last layer, no normalisation of interaction outputs) could also matter.

5. What each step gained

Step	What it models	Gain on this data	Task
Bias baseline	who rates high, which items are liked	18.7% below global mean (val RMSE)	01
Neighbourhood CF	similarity between users or items	5.6% below bias baseline (val RMSE)	01
Matrix factorisation	latent factors; dot product of user and item vectors	6.2% below bias baseline (test RMSE); 0.6% ahead of item-item	01
Embeddings + MLP	non-linear patterns across many sparse fields	PR-AUC 3.2× chance on val, 1.9× on test	02
DLRM	explicit dot products between all field pairs	no measurable gain over Task 02	03

The tasks use different data and metrics, so these numbers compare only as gains over a simple reference, not as scores. The pattern is consistent: in both rating prediction and CTR, most of the improvement comes from simple structure (bias terms in Task 01, learned embeddings in Task 02), and each more elaborate model adds little at these dataset sizes.

6. Limitations

- One split per task; three seeds per configuration; small sweep grids, with no learning-rate or weight-decay search for Tasks 02 and 03.
- Task 02/03 validation and test hold only 256 and 335 clicks, so PR-AUC differences under about 0.01 are not meaningful.
- The Task 02 validation-to-test drop is unexplained.
- The DLRM ablation changes the top MLP's input size as well as removing interactions; seed-pairing across different architectures is loose.
- Task 01 ranking evaluation ranks by predicted rating, a protocol known to favour popularity; the user-user ranking failure was not diagnosed.
- The DLRM architecture was implemented from summaries of the paper rather than a line-by-line reading of it.

7. Reproducibility

Task 01: Kaggle notebook (numpy implementation; does not use the accelerator). Seeds: split 42; MF 42, 43, 44 (final model 42). Tasks 02 and 03: Google Colab, CPU only; Python 3.13.15, numpy 2.1.3, torch 2.11.0+cpu, scikit-learn 1.6.1, matplotlib 3.10.0. Seeds: split 42; sweeps 42, 43, 44; selected models 42. Notebooks, per-task READMEs and timestamped LOG.md files are in the submission repository under `mandatory-tasks/recommender-systems-cf-to-dlrm/RecommendationTasks/`. Package versions for each task are recorded in its README.

References

- F. M. Harper and J. A. Konstan. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems*, 5(4), 2015.
- Y. Koren, R. Bell and C. Volinsky. Matrix Factorization Techniques for Recommender Systems. *IEEE Computer*, 42(8), 2009.
- M. Naumov et al. Deep Learning Recommendation Model for Personalization and Recommendation Systems. arXiv:1906.00091, 2019.

- WebClub NITK. Intelligence-SIG-Recs-2026 task repository,
github.com/WebClub-NITK/Intelligence-SIG-Recs-2026.

AI usage: AI tools were used for explanation and debugging help; all code and results were run and checked by me.